

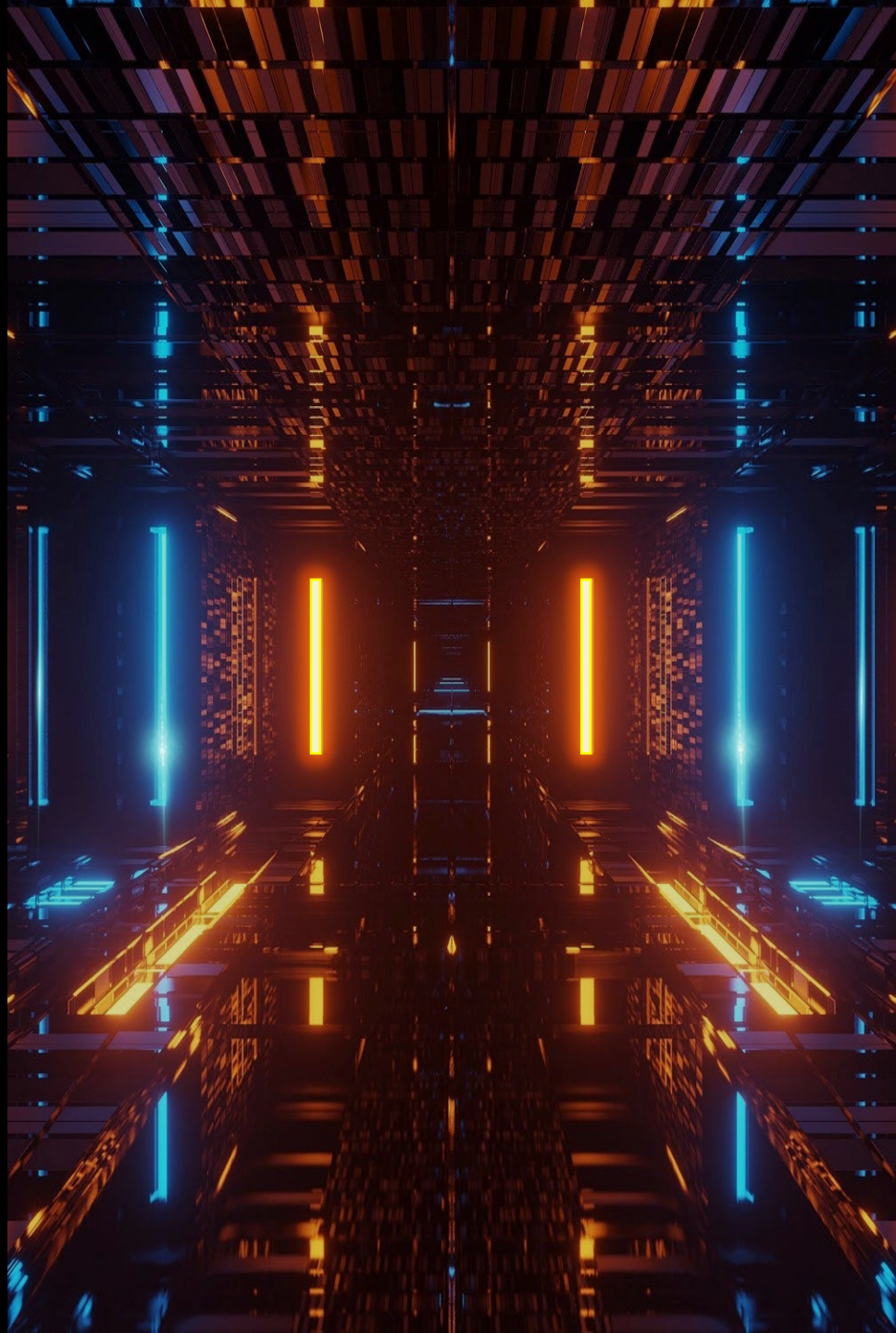
2026 04.28 —————> 06.19

数据科学与大数据 技术的数学基础

Mathematical Foundations of
Data Science

罗霄

317834051@qq.com



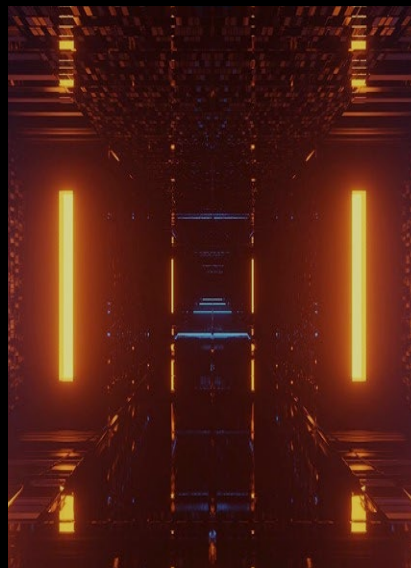
数据科学与大数据技术的数学基础

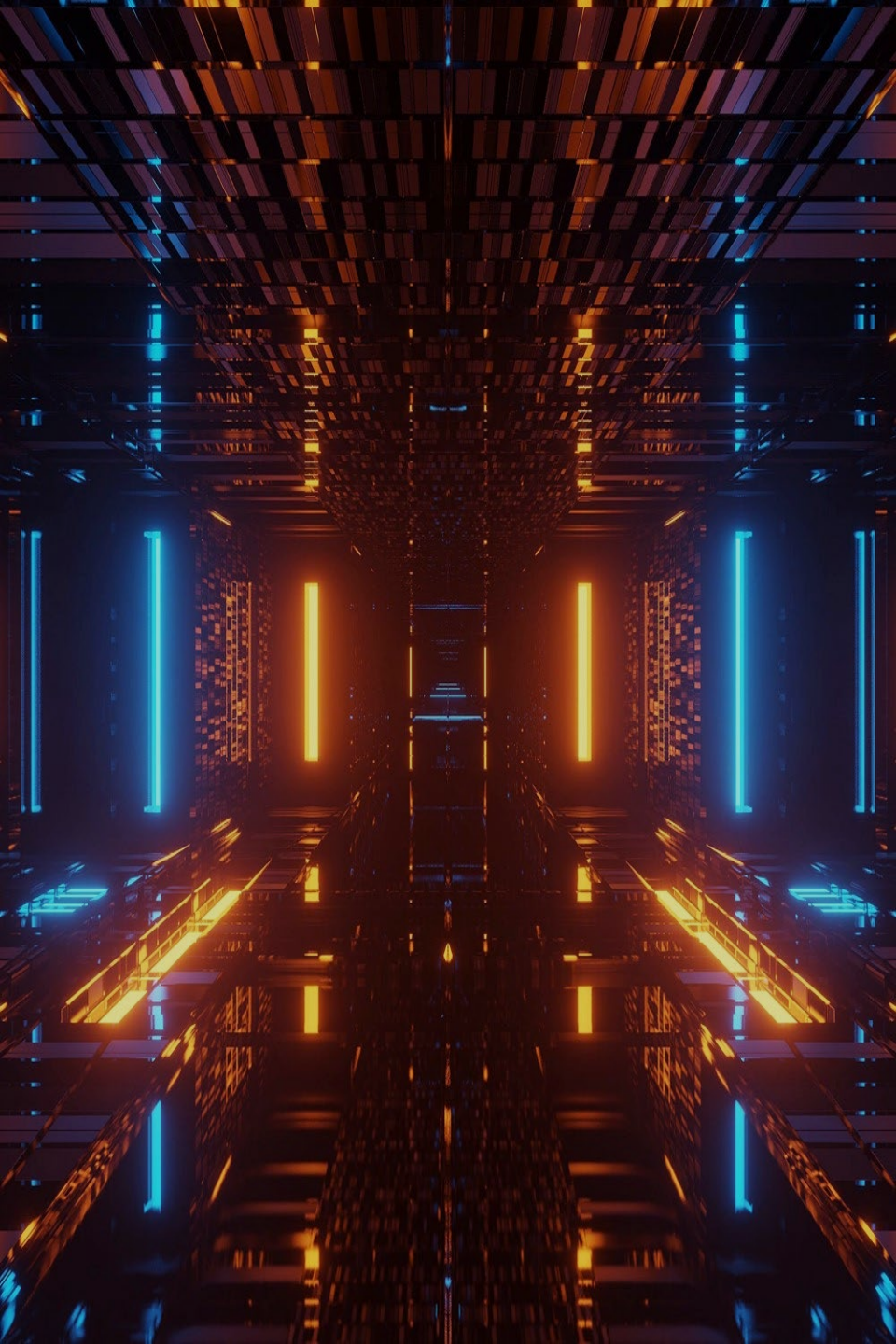
Mathematical Foundations of Data Science

/03

泛化与正则(2)

Generalization and
Regularization





Contents

1. What is generalization
2. How to understand generalization
3. Dataset classification
4. PAC (Probably Approximately Correct)
5. Regularization

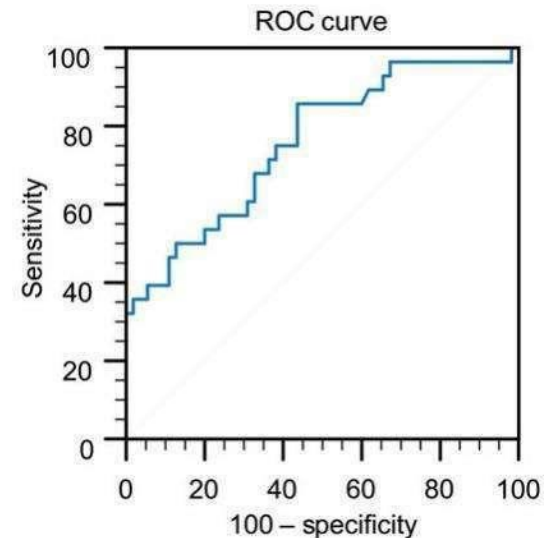
PS ROC & AUC

?

For the 0,1 classification problem, some classifiers often get results with labels other than 0,1, such as the neural grid, and get classification results such as 0.5,0.8. In this case, we artificially take a **threshold value**, such as 0.4, then those less than 0.4 are 0 classes, and those greater than or equal to 0.4 are 1 classes, and we can **get a classification result**. Similarly, the threshold value can be 0.1, 0.2, etc. If we take different thresholds, we can get different classification results.

Receiver Operating Characteristic (ROC)

So we need a method to **check the accurate classification ability of the threshold**



1. First invented by electronic and radar engineers in World War II to detect enemy carriers on the battlefield, i.e., **signal detection theory**;
2. Later used in **psychology** for the perceptual detection of signals;
3. Now used in machine learning to judge the goodness of classification and detection results.

ROC curves are very important and common statistical analysis methods.

Real categories	Predicted categories	
	Positive Prediction	Negative Prediction
Positive Class	TP	FN
Negative Class	FP	TN

$$\text{True positive rate TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{False positive rate FPR} = \frac{\text{FP}}{\text{TN} + \text{FP}}$$

ROC curves combine sensitivity and specificity in a graphical way and can accurately reflect the relationship between the **specificity (FPR: 1-specificity)** and **sensitivity (TPR)** of an analytical method, and are a **comprehensive representation of the accuracy of the test**.

PS ROC & AUC

How to plot ROC curves

真正例率: $TPR = \frac{TP}{TP+FN}$

假正例率: $FPR = \frac{FP}{TN+FP}$

1. Given m positive examples and n negative examples, rank them from highest to lowest according to the learner's prediction

2. First set the classification threshold to the maximum so that all examples are predicted to be counterexamples, when both TPR and FPR are 0.

3. Mark a point at (0, 0)

4. The classification threshold is set to the predicted value of each sample in turn, i.e., each example is divided into positive examples in turn

5. Let the previous coordinate be (x,y). If the current instance is a true positive, the corresponding marked point is (x,y+1/m); if it is a false positive, the marked point is (x+1/n, y).

For example

1. There are 10 sample examples, 5 positive examples and 5 negative examples. There are two learners A,B, each predicting the 10 examples, and the results are sorted from highest to lowest according to the predicted values (not specifically listed here) as follows:

A: [N P P P N N P P N N]; B: [N P N N N P P P P N];

2. A: [N N N N N N N N N N] B: [N N N N N N N N N N]

3. TP=FP=0, TRP=FPR=0, x=y=0

4. A: [N N N N N N N N N N] → A: [P N N N N N N N N N]

The first one is a false positive, $x_1=0+1/5=0.2$, $y_1=0$

A: [P N N N N N N N N N] → A: [P P N N N N N N N N]

The second one is a true positive, $x_2=0.2$, $y_2=0+1/5=0.2$

A: [P P N N N N N N N N] → A: [P P P N N N N N N N]

3rd true positive, $x_3=0.2$, $y_3=0.2+1/5=0.4$

A: [P P P N N N N N N N] → A: [P P P P N N N N N N]

4th true positive, $x_4=0.2$, $y_4=0.4+1/5=0.6$

Inst#	A	Class	Score
1		N	0.9
2		P	0.88
3		P	0.7
4		P	0.65
5		N	0.6
6		N	0.53
7		P	0.5
8		P	0.3
9		N	0.25
10		N	0.2

PS ROC & AUC

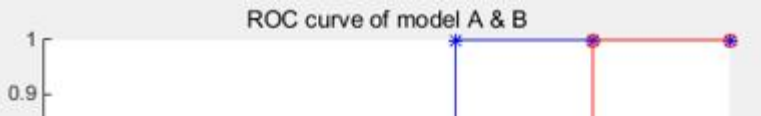
1. There are 10 sample examples, 5 positive examples and 5 negative examples. There are two learners A,B, each predicting the 10 examples, and the results are sorted **from highest to lowest** according to the predicted values as follows:

A: [N P P P N N P P N N]; B: [N P N N N P P P P N];

Then connect the points in turn and follow the plotting process to get the ROC curve points corresponding to the learner

A: $y:[0,0,0.2,0.4,0.6,0.6,0.6,0.8,1,1,1]$ The blue color is the ROC curve of learner A, which
 $x:[0,0.2,0.2,0.2,0.2,0.4,0.6,0.6,0.6,0.8,1]$ contains the curve of B, indicating that it has better
B: $y:[0,0,0.2,0.2,0.2,0.2,0.4,0.6,0.8,1,1]$ performance.

$x:[0,0.2,0.2,0.4,0.6,0.8,0.8,0.8,0.8,0.8,1]$ In addition, if two curves cross, the area enclosed by the curve (AUC) needs to be calculated to evaluate the superiority of performance.



1.The ROC curve makes it easy to identify the discriminatory ability of a test at any given threshold value.

2.Select the optimal diagnostic threshold. The closer the ROC curve is to the upper-left corner, the higher the accuracy of the test. The point on the ROC curve closest to the upper-left corner (i.e., where the Youden index, $TPR-FPR$, is maximized) represents the best threshold with the fewest errors, minimizing the total number of false positives and false negatives.

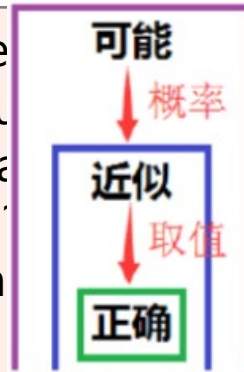
3.When comparing two or more diagnostic methods for the same algorithm, the ROC curves of each test can be plotted on the same coordinate system to visually identify the superior one—the ROC curve that lies closer to the upper-left corner indicates the most accurate performance of the test. Alternatively, the comparison can be performed by calculating the area under the ROC curve (AUC) for each test; the test with the largest AUC has the best diagnostic value.

3.4 PAC (Probably Approximately Correct)

?

As one goes through the example one may have doubts about several questions:

- What kind of algorithm can learn data?
- What are the difficulties of learning?
- How many samples used to train the model?
- Is there a universal model to learn?



PAC learning theory enables you to see why a hypothesis (model or function) that has learned a training sample is guaranteed to be valid on data other than the training sample.

PAC: probably approximately correct 概率近似正确 learning framework.

The theorem says that for a given classifier, when the **data size is sufficiently large**, then **zero training error**, necessarily **leads to small true errors**. This provides a theoretical proof of the significance of zero training error.

PAC-learning can be divided into two parts:

Approximately Correct:

A hypothesis that $h \in H$ is "approximately correct" means that its **generalization error $R(h)$** is less than a bound ϵ , ϵ is generally a number tending to 0. If $R(h)$ **is larger, the model cannot be used to make correct "predictions"**.

Probably:

A learning algorithm **A has the "possibility" of learning such an "approximately correct" assumption with probability $1 - \delta$** .

A PAC-Learnable algorithm is one algorithm that is able to **learn an approximately correct assumption h from a reasonable amount of training data in polynomial time**.

3.4 Mathematical concepts about PAC

Do you remember

The algorithm first trained on the **training set**



Obtain a **model**



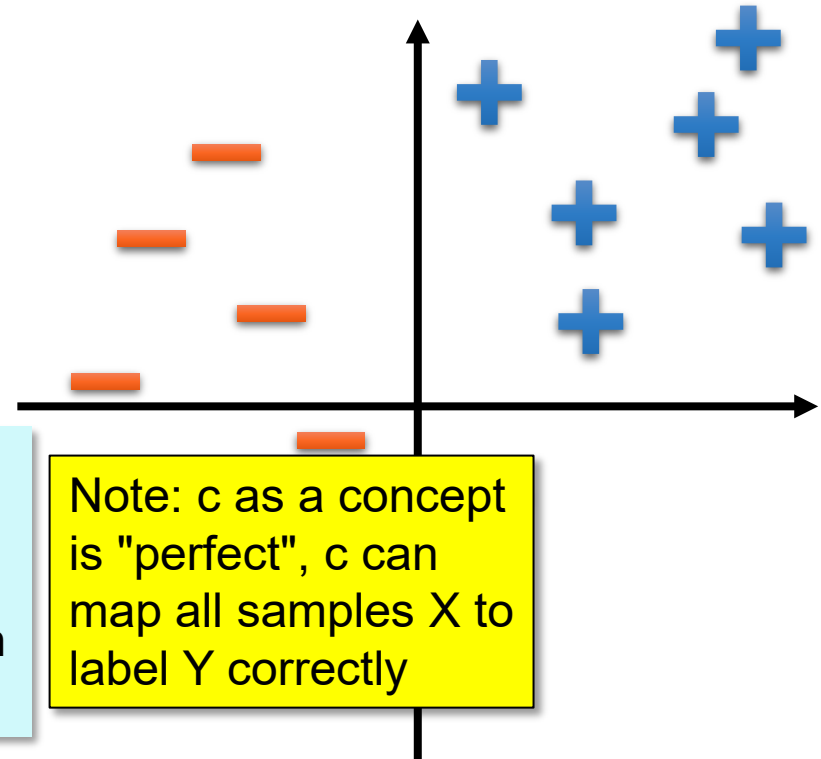
Evaluate performance on the **test set**

X: sample set $\{+, -\}$, input space

Y: Label or target value set, $Y=\{0,1\}$

Since **binary classification** is the most common in life and in machine learning, we will discuss the binary classification problem in this example

As shown in the figure, **what algorithm would you design** so that the **+** label is **1** and the **-** label is **0**?



1) **A mapping from training sample to the correct target**, called a '**concept**', is expressed by the symbol **c**. There is a mapping such that **c(x) = y**, which is only a certain result and not a set.

2) **The set of all mappings we wish to learn**, from the training samples to the correct destination, is the '**concept class**', denoted by the symbol **C**. Obviously, not all of the mappings we have to learn, we just have to learn some specific concept **c**.

3.4 Mathematical concepts about PAC

?

An important problem in machine learning is the problem of **predicting the distribution of samples**, which requires that the new samples (test samples) and the existing samples (training samples) must be predicted to be **independently and identically distributed (i.i.d.)** among all the samples, i.e., all of the samples are **in the same unknown distribution D** .

3) Now we take all the **concepts (mappings)** that can be **mapped from X samples to Y labels** and form a **function set or model set** called 'hypothesis space', which is denoted by the symbol **H** .

Note that **H and C may not be consistent**. Because, as we said earlier, not all $X \rightarrow Y$ mappings we want to learn, just as not all GCE exercises we want to brush up on.

4) The learner is able to obtain a **sample set S** , $S = \{(x_1, c(x_1)), \dots, (x_m, c(x_m))\}$, as the sample labels are generated by the target concept c (a perfectly correct mapping from X to Y , imagine labeling the image manually).

Then the learner task is to **use S to find a hypothesis $h_s \in H$** (the computer learns to mimic the set of samples and infer the model backwards), so that it has **as small a generalization error as possible for the target concept c** (the smaller the difference from a perfect mapping, the better).

5) Assume that the **generalization error or true error of $h \in H$** is denoted as **$R(h)$** .

3.4 Mathematical concepts about PAC Generalization error & Empirical error

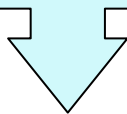
5) **Generalization error or true error of $h \in H$** is denoted as **$R(h)$** .

Hypothesis $h \in H$, target concept $c \in \mathcal{C}$, and potential distribution $\mathbf{D}$. The **generalization error (risk of h)** for the hypothesis set $h_s \in H$ (vs. target concept c) under distribution $\mathbf{D}$:

$$R(h) = \mathbb{P}_{x \sim \mathbf{D}}[h(x) \neq c(x)] = \mathbb{E}_{x \sim \mathbf{D}}[\mathbf{1}_{h(x) \neq c(x)}]$$

where $\mathbf{1}_\omega$ is the **indicator function**, the **inequality holds** the return value of **1**, and the inequality does **not hold** returns **0**.

Since both the **distribution $\mathbf{D}$** and the **target concept c** are unknown, we cannot obtain the **generalization error** of the hypothesis set **directly**. So we can **measure the empirical error** of the hypothesis set **on the labeled sample set S** .



6) **Empirical error of $h_s \in H$** is denoted as **$\hat{R}(h)$** .

Hypothesis $h \in H$, target concept $c \in \mathcal{C}$, and a **sample set S** , $S = \{(x_1, c(x_1)), \dots, (x_m, c(x_m))\}$. The **empirical error (empirical risk of h)** for the hypothesis set $h_s \in H$ (vs. target concept c) under distribution $\mathbf{D}$:

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}_{h(x_i) \neq c(x_i)}$$

3.4 Mathematical concepts about PAC Generalization error & Empirical error

5) **Generalization error or true error of $h \in H$** is denoted as **$R(h)$** .

$$R(h) = \mathbb{P}_{x \sim D}[h(x) \neq c(x)] = \mathbb{E}_{x \sim D}[\mathbf{1}_{h(x) \neq c(x)}]$$

6) **Empirical error of $h_s \in H$** is denoted as **$\hat{R}(h)$** .

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}_{h(x_i) \neq c(x_i)}$$

Since the sample set S is an independent identically distributed (**i.i.d.**) sample of D , the **empirical risk is the average error of the sample set S** . The **expectation of the empirical error** over the distribution D = the **generalization error** of the sample.

$$\mathbb{E}_{S \sim D^m}[\hat{R}(h)] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{S \sim D^m}[\mathbf{1}_{h(x_i) \neq c(x_i)}] = \mathbb{E}_{x \sim D}[\mathbf{1}_{h(x) \neq c(x)}] = R(h)$$

So the **empirical error expectation** is equal to the generalization error

This gives us an indication of the direction of the computer's efforts to build models from sample sets
→ **trying to reduce empirical errors**

$$E[\hat{R}(h)] = R(h)$$

3.4

Under what conditions we said PAC learnable?

—Under what conditions can an algorithm/model label an image like a human

Definition 1: PAC Identify: For $0 < \epsilon, \delta < 1$, all $c \in \mathcal{C}$ and distribution $\mathbf{D}$, if there exists a **learning algorithm** $\mathcal{L}$ whose **output** $h \in H$, if satisfying

$$P(R(h) \leq \epsilon) \geq 1 - \delta$$

Then the **learning algorithm** $\mathcal{L}$ can **PAC identify** the concept class $\mathcal{C}$ from the hypothesis space H

That is, the **probability** that the **generalization error on the hypothesis set** h of the sample $\mathbf{S}$ **is less than** ϵ is **greater than or equal to** $1 - \delta$, which means that **the algorithmic model** is guaranteed to **have a small generalization error** in a large number of cases.

Definition 2: PAC Learnable: Let m denote the number of samples obtained from independent co-distribution sampling in distribution $\mathbf{D}$, $0 < \epsilon, \delta < 1$, the complexity of the concept $c \in \mathcal{C}$ is denoted as $\text{size}(c)$.

For all distributions $\mathbf{D}$, if there exists a **learning algorithm** $\mathcal{L}$ and a polynomial function $\text{poly}(1/\epsilon, 1/\delta, \text{size}(x), \text{size}(c))$, the following equation holds **for any sample size** m satisfying $m \geq \text{poly}(1/\epsilon, 1/\delta, \text{size}(x), \text{size}(c))$.

$$\Pr_{S \sim D^m}(R(h_S) \leq \epsilon) \geq 1 - \delta$$

Then the **learning algorithm** $\mathcal{L}$ can **PAC identify** the concept class $\mathcal{C}$ from the hypothesis space H , the concept class $\mathcal{C}$ is said to be **PAC-learnable** for the hypothesis space H . Sometimes it is also referred to as the concept $\mathcal{C}$ is PAC-learnable.

You say PAC learnable, so how long will it take? -- Consider the time complexity

3.4

Under what conditions we said PAC learnable?

—Consider the time complexity and Sample Complexity

Definition 3: PAC Learning Algorithm: A concept class $\mathcal{C}$ is said to be **efficiently PAC-learnable**, if the learning algorithm $\mathcal{L}$ makes it PAC-learnable and the **running time of $\mathcal{L}$ is within poly** $(1/\epsilon, 1/\delta, \text{size}(x), \text{size}(c))$, $\mathcal{L}$ is the **PAC learning algorithm** for concept class $\mathcal{C}$.

If the learning algorithm **takes a constant time to process each sample**, then the **time complexity is equivalent to the sample complexity**

Definition 4: Sample Complexity: The **smallest m** of $m \geq \text{poly}$ $(1/\epsilon, 1/\delta, \text{size}(x), \text{size}(c))$ required to satisfy the PAC learning algorithm $\mathcal{L}$ is called the **sample complexity** of the learning algorithm $\mathcal{L}$.

- $1 - \epsilon$ denotes the accuracy, $1 - \delta$ denotes the confidence level
- The PAC learning framework is a distribution-free model; it makes no particular assumptions about the distribution D

The training samples and test samples are generated from the same distribution D — this is a necessary condition for generalization to be possible. The PAC learning framework considers the learnability of a concept class $\mathcal{C}$, rather than the learnability of a specific concept. The concept class $\mathcal{C}$ is known, while the target concept c is unknown.

$$P(R(h) \leq \epsilon) \geq 1 - \delta$$

*有限假说集的保证：一致/非一致的情况，请参考《机器学习》（周志华）版本

3.5 Empirical Risk Minimization

Risk in Empirical Risk Minimization

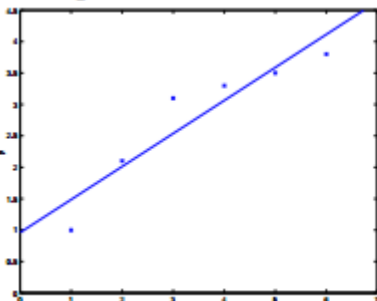
In the "risk minimization principle" of **machine learning**, "**risk**" refers to the **average error** between the **prediction result of the prediction function** and the **result of the labeled data**.

If **X** refers to the sample, **Y** refers to the **true classification** of the sample, and **f(X)** refers to the **predicted classification** of the sample **estimated by the model**, we need to **compare** the difference between the **predicted classification f(X)** and the **true classification Y**, and use it as the good or bad prediction of the model.

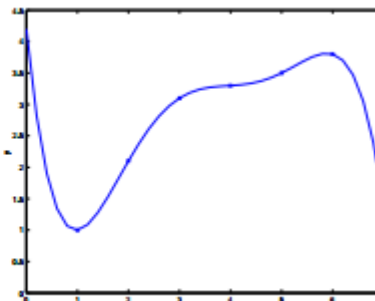
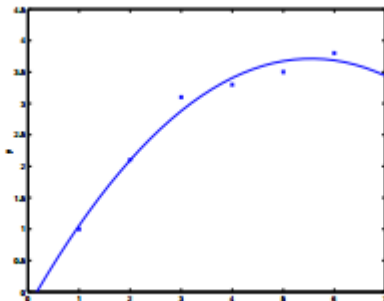
Bias or variance

In machine learning, the trade-off between bias and variance is a problem that needs to be focused on.

Taking the regression problem as an example, the goal of machine learning is to **obtain a model** from the **training set** so that it can **make predictions on the test set**. At this point, both the training and test sets are samples of the **D** distribution.



underfitting,
high bias



overfitting,
high variance

When we talked about linear regression, we discussed whether to fit the data by using the simple linear assumption $y = \theta_0 + \theta_1 x$ or more **complex** assumption $y = \theta_0 + \theta_1 x + \dots + \theta_5 x^5$?

3.5 Empirical Risk Minimization (how to use)

Loss function L

Express the degree of **difference** between the **prediction** and the **actual data**

Measuring **how well** the model predicts **once**

Cost Minimization is based on the defined **cost function** to **minimize** the overall loss function of the training set

Empirical risk function R

$$R_{emp}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

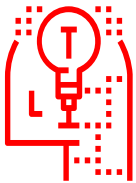
Measuring **how well** the model predictions in **an average sense**

Empirical risk minimization is to determine the parameters of the model such that the **empirical risk function** is **minimized**

There are **many** kinds of **loss functions**, and in the process of using them, the **smaller** the **loss function** value is, the **better** the prediction result is, and the better the **model**.

The inputs to the model are random variables that follow a joint distribution, and the **expectation of the loss function** (the **average value taken by the loss function**), is referred to as the **expected risk**.

Since the joint distribution unknown, the **expected risk** cannot be calculated directly and can only be approximated as the **average loss of $f(X)$** with respect to the **training data set** as **empirical risk**.

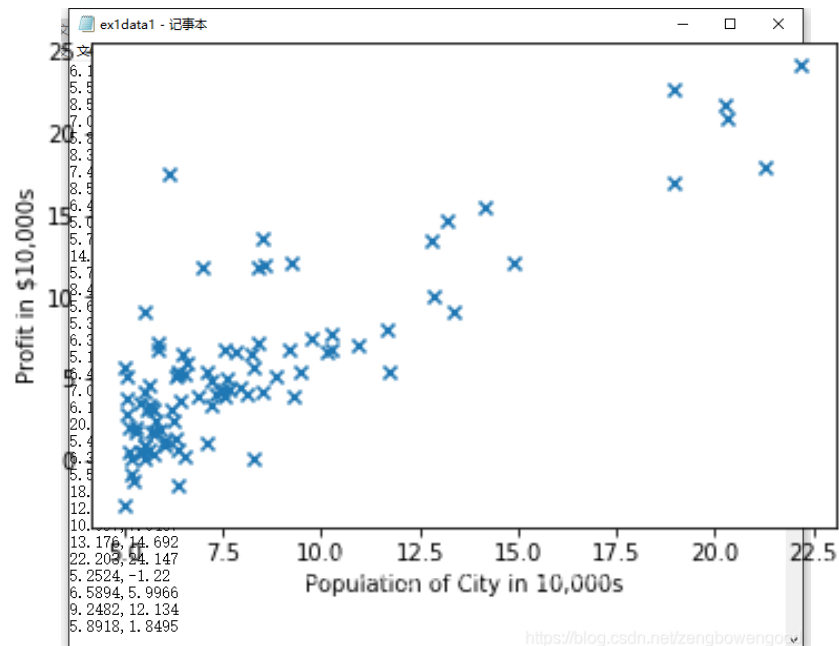


On the previous page we tried to compare the linear model $y = \theta_0 + \theta_1 x$ and the higher-order model $y = \theta_0 + \theta_1 x + \dots + \theta_5 x^5$ to fit the data.

In a high-dimensional feature matrix, neither of these two approaches is clearly very desirable. Here we first borrow the **linear regression model** and **binary classification** to demonstrate that **empirical risk minimization** does lead to **smaller generalization errors**.

1. Significance of the Univariate linear regression model

Unary linear regression is a method to analyze the linear correlation of only one independent variable (independent variable x) and dependent variable y .



ex1data1.txt has two columns of data, the **first column** is the **independent variable (x)** and the **second column** is the **dependent variable (y)**, both are continuous variables with a **dimension of 97×2**

1) First, we are going to do a preliminary exploration of the data: **plot a scatter plot of the dependent and independent variables**, and observe **what pattern** the scatter plot fits.

2) From the scatter plot, we see that the independent and dependent variables have a relatively good **linear relationship**, so the hypothesis can be made, assuming a function of **$h_{\theta}(x) = \theta^T x$**

Linear regression

A statistical analysis method that uses regression analysis in mathematical statistics to determine the **degree of interdependence between** two or more **variables**.

- A regression analysis that includes **only one independent variable and one dependent variable**, and the relationship between them can be approximated by a **straight line**, is called a **univariate linear regression** analysis.
- If **two or more independent variables** are included in the regression analysis and the relationship between the dependent variable and the independent variable is **linear**, it is called **multiple linear regression** analysis.

In linear regression, the data are **modeled** using a **linear predictive function** and the **unknown model parameters** are estimated from the **data**. These models are called **linear models**.

For a sample with n features, its **regression results** can be written in an **equation** familiar to almost everyone.

$$\hat{y}_i = \omega_0 + \omega_1 x_{i1} + \dots + \omega_n x_{in}$$

$x_{i0} = 1$, If we have m samples, the regression results can be written as:

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \dots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ 1 & x_{31} & x_{32} & \dots & x_{3n} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} * \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \dots \\ \omega_n \end{bmatrix}$$

$$\hat{\mathbf{y}} = \mathbf{X}\omega$$

We need to find the unknown ω parameter (note that this is the time to minimize the **sum of squares** of the residuals between the true and predicted values)

PS Linear regression

$$f(x) = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + b$$

$$f(x) = \theta^T x + b$$

Learning =
Representation **Best fit straight line**
+ Evaluation **Least square method**
+ Optimization



Goal: $y = \theta_0 + \theta_1 x + \varepsilon$

Where θ_0 and θ_1 are model coefficients, ε It is a random error term, **let's ignore it first.**

m: Number of instances in the training set
(number of samples)

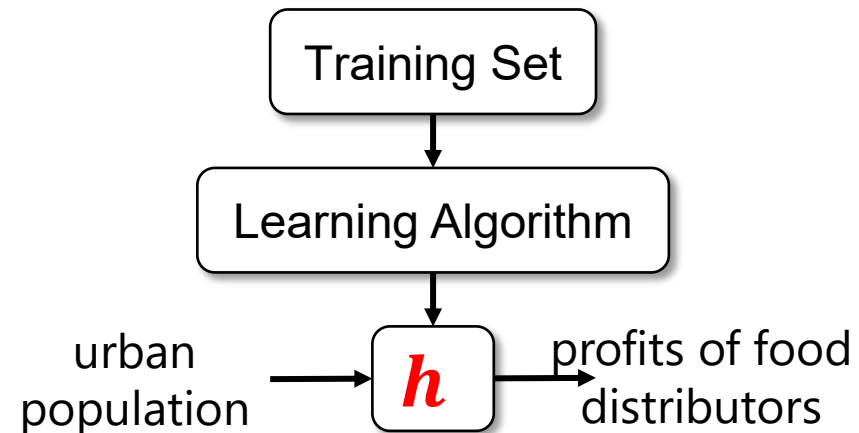
x: Features/input variables (referring to
urban population here)

y: Target variable/output variable
(referring to profits of food distributors
here)

(x,y): Examples in the training set

$(x^{(i)}, y^{(i)})$: The i-th observation instance

h : Solutions or functions for learning
algorithms, also known as hypothesis



$$h_{\theta}(x) = \theta_1 x_1 + \theta_0$$

PS Linear regression: Cost function

For a univariate linear function, each pair of parameters can yield a method of drawing a straight line

$$h_{\theta}(x) = \theta_1 x_1 + \theta_0$$

Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

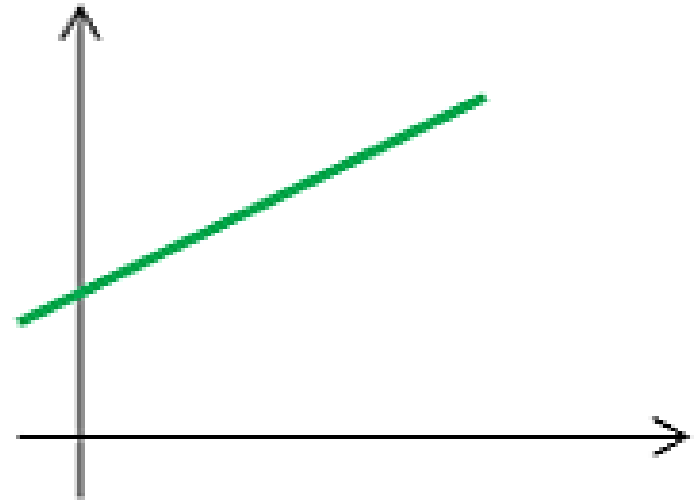
Parameters:

$$\theta_0, \theta_1$$

Cost function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

Goal: $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$



Least square method

Cost function

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

Note: The **cost function** is defined as the average of the sum of errors for all samples on the entire training set

The **loss function** is defined on a single training sample

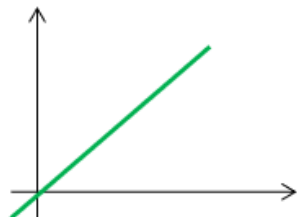
Our ultimate goal is to minimize the cost function

Simplified

$$h_{\theta}(x) = \theta_1 x$$

Parameters:

$$\theta_1$$

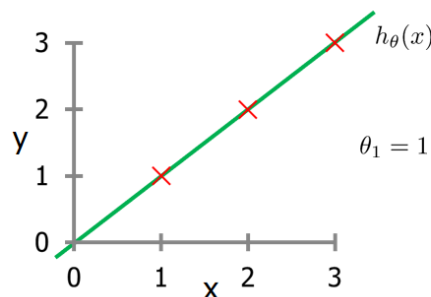


$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

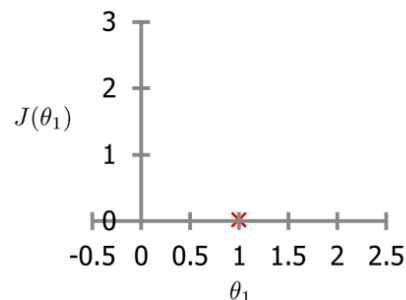
$$\underset{\theta_1}{\text{minimize}} J(\theta_1)$$

The assumed function corresponding to the parameter **when $J(\theta_1)$ takes its minimum value** is the best fitting straight line for the data

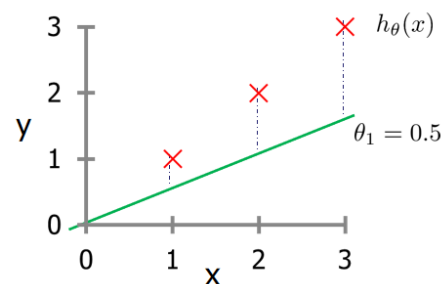
$h_{\theta}(x)$
(for fixed θ_1 , this is a function of x)



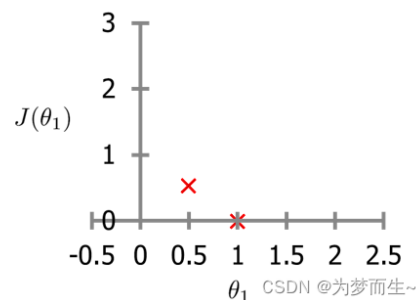
$J(\theta_1)$
(function of the parameter θ_1)



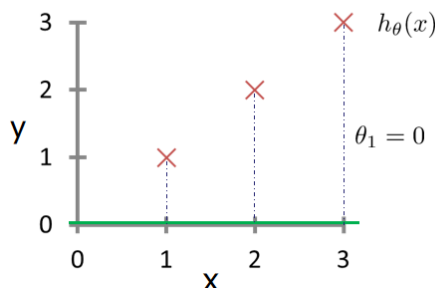
$h_{\theta}(x)$
(for fixed θ_1 , this is a function of x)



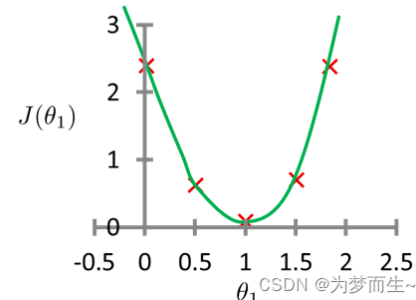
(function of the parameter θ_1)



$h_{\theta}(x)$
(for fixed θ_1 , this is a function of x)



$J(\theta_1)$
(function of the parameter θ_1)



Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Parameters:

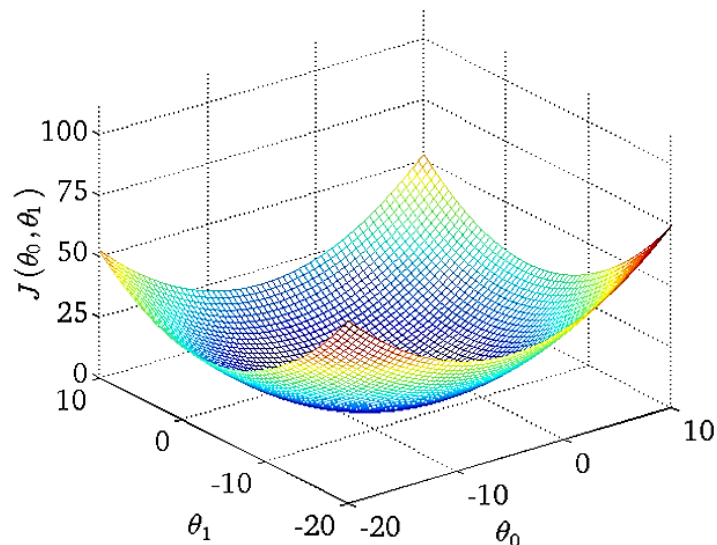
$$\theta_0, \theta_1$$

Cost function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

Goal: $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Using the same method as above, traverse each pair of parameters θ_0, θ_1 , and find the parameter that minimizes the cost function

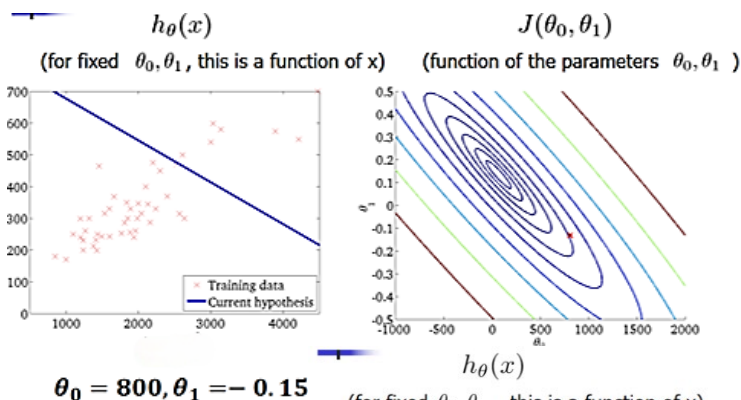


As shown in the left Figure 1:

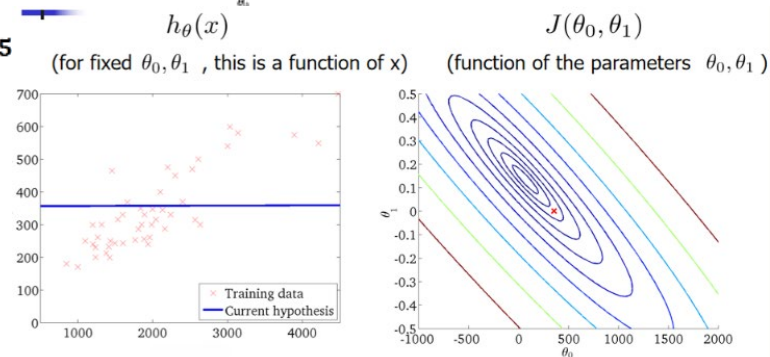
When we select a pair of parameter values, $\theta_0 = 800$, $\theta_1 = -0.15$, and substitute them into $h_{\theta}(x)$, we draw the blue line (observe how far the discrete red points y are from the blue line at the corresponding x positions).

The difference between the theoretical value and the actual value for each sample is accumulated according to the cost function, then divided by $2m$, yielding $J(\theta_0, \theta_1)$.

The right figure is a planar view; look carefully at the contour lines in the right figure. The red X marks the location corresponding to the parameters θ_0, θ_1 , and the contour lines indicate the value of J .



$$\theta_0 = 800, \theta_1 = -0.15$$

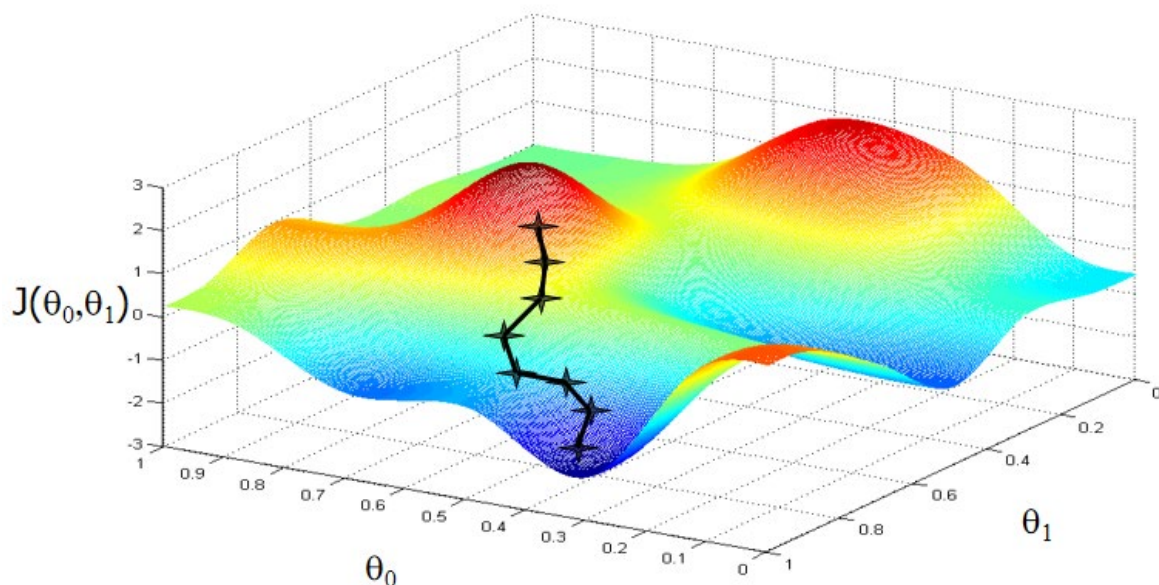


$$\theta_0 = 360, \theta_1 = 0$$

Learning =
Representation **Best fit straight line**
+ Evaluation **Least square method**
+ Optimization **Gradient descent**

For example, the gradient descent method involves finding the **shortest path from the mountaintop to the lowest point in the valley**. During this process, each step will follow the gradient direction of the current point (i.e. the slope of the function at that point), gradually approaching the lowest point.

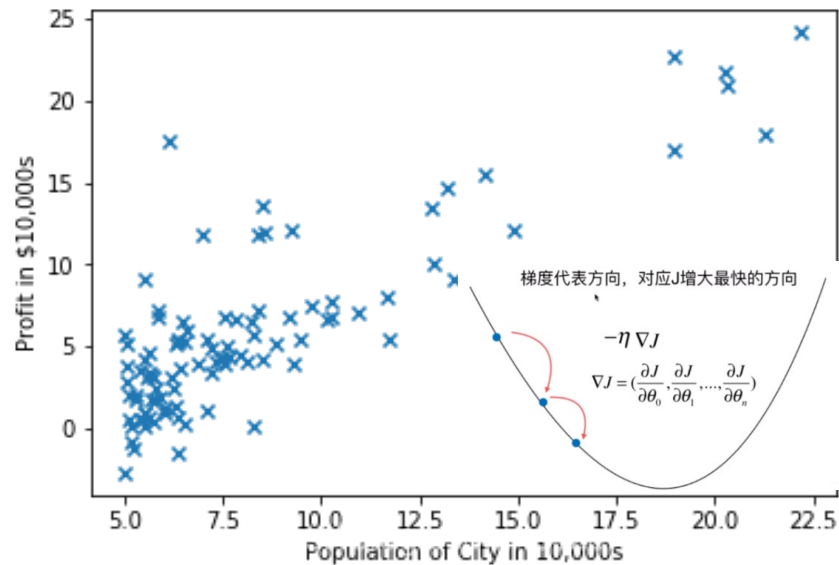
Gradient descent is a type of iterative method that can be used to solve least squares problems (both linear and nonlinear). Gradient descent is one of the most commonly used methods for solving model parameters, i.e. **unconstrained optimization problems**, in machine learning algorithms



Combining gradient descent with linear regression
(key lies in **taking partial derivatives of the cost function**)

PS Linear regression

1. Significance of the Univariate linear regression model



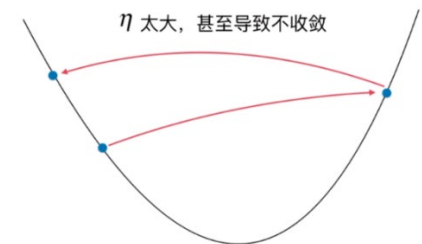
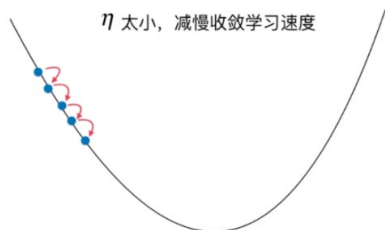
ex1data1.txt has two columns of data, the **first column** is the **independent variable (x)** and the **second column** is the **dependent variable (y)**, both are continuous variables with a **dimension of 97*2**

1) First, we are going to do a preliminary exploration of the data: **plot a scatter plot of the dependent and independent variables**, and observe **what pattern** the scatter plot fits.

2) From the scatter plot, we see that the independent and dependent variables have a relatively good **linear relationship**, so the hypothesis can be made, assuming a function of **$h_{\theta}(x) = \theta^T x$**

3) We use the **loss function** (the difference between the predicted and true values) and consider that when using the **squared loss function directly** $L = (Y - f(X))^2$, using the **gradient descent method** will introduce **m** , leading to the gradient size being correlated with the sample size **m** . The larger the number of samples, the larger the gradient will be. Obviously unreasonable. We **divide the loss function by 2m** (m also works) to get the following equation:

$$\text{Training error } J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$



Combining gradient descent with linear regression
(key lies in **taking partial derivatives of the cost function**)

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (\text{for } j = 0 \text{ and } j = 1)$$

}

learning rate

CSDN @为梦而生~

Among them, α is the learning rate, which determines how much step we take down along the direction that can minimize the cost function. In batch gradient descent, we subtract the learning rate from all parameters and multiply it by the derivative of the cost function each time.

Correct: Simultaneous update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

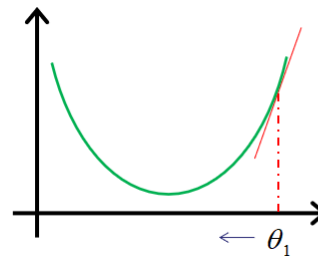
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_0 := \text{temp0}$$

$$\theta_1 := \text{temp1}$$

CSDN @为梦而生~

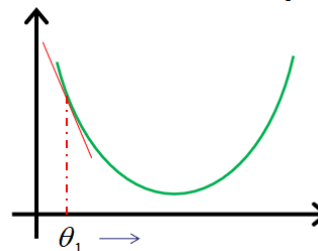
The purpose of partial differentiation is to determine the direction of descent



$$\theta_1 = \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

≥ 0

CSDN @为梦而生~



$$\theta_1 = \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

≤ 0

CSDN @为梦而生~

Correct: Simultaneous update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_0 := \text{temp0}$$

$$\theta_1 := \text{temp1}$$

CSDN @为梦而生~

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2 \end{aligned}$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

CSDN @为梦而生~

repeat until convergence {

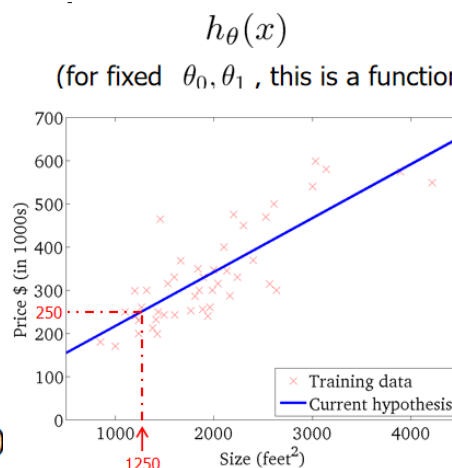
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

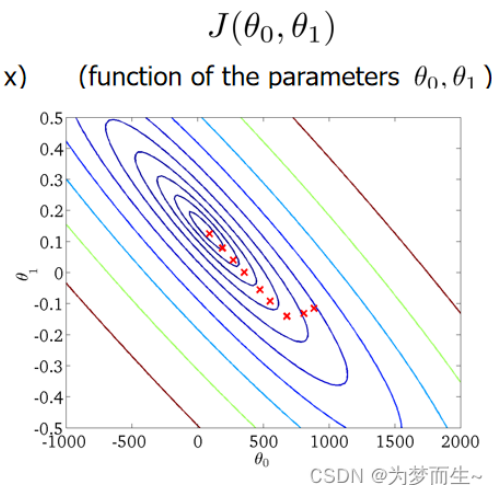
}

update
 θ_0 and θ_1
simultaneously

The purpose of partial differentiation is to determine the direction of descent



($\theta_0 = 900, \theta_1 = -0.1$)



CSDN @为梦而生~

PS Linear regression

1. Significance of the Univariate linear regression model

In addition, because the references from different sources, so this section the **corner of x** represents **from which sample** may sometimes **be in the upper right corner and sometimes in the lower right corner**, we mainly observe the **range of values of i to determine** (the feature dimension is generally different from the number of samples in the range of values)

Population of City in 10,000s

1) First, plot a scatter plot of the dependent and independent variables

2) assuming a function of $\mathbf{h}_{\theta}(\mathbf{x}) = \theta^T \mathbf{x}$

3) We use the **loss function**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

4) Solving for **empirical risk minimization**:

When solving for the minimum of the loss function, we can use the **least squares method**. But this has some drawbacks: **when the amount of data is large, the computation becomes larger**, and the efficiency is not reflected.

* **Gradient descent method** is a more general method for solving the optimal solution of the objective function.

Gradient: In calculus, we find the **partial derivatives** of the parameters of multivariate functions, and write out the partial derivatives of each parameter in the form of a vector, which is the gradient. It is easier to find the maximum value of the function along the direction of the gradient vector, and easier to find the **minimum value along the opposite direction of the gradient vector**.

Therefore we use the gradient descent method, when the gradient is such that the **partial derivative of J for each dimension of θ : $\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m (y^i - h_{\theta}(x^i)) x_j^i$** , repeat $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$

(选择学习率 α <步长>, 经过多次迭代, 就可以求得经验风险最小值, 即求出合适的 θ 系列参数)

PS Linear regression

2. From Univariate linear regression to linear regression

?

Lemma: If we do not want to discuss the relationship between urban population and gross GDP, but rather when we explore the relationship between weight and gender, height, etc., i.e., when we explore the relationship between one dependent variable and multiple independent variables, we should consider using multiple linear regression.

Many diseases in medical research have multiple etiologies. For example, blood glucose changes in diabetic patients may be influenced by insulin levels, glycated hemoglobin, total serum cholesterol, triglycerides and other indicators, etc. Multiple linear regression can be used to explore the influencing factors of blood glucose in diabetic patients.

Assume: the sample is denoted by x and the k th sample in the data set is x_k . There are d feature in a sample, and we represent a sample by a column vector as $x_k = (x_k^1; x_k^2; \dots; x_k^d)$. ω is the weight to be learned. Since each sample has d features, ω is a d -dimensional column vector, denoted as $\omega = (\omega^1; \omega^2; \dots; \omega^d)$. With ω and b determined, the model is determined down to $\hat{y} = \omega^T x + b$. We use the mean square error to measure the performance of the model, m is the total number of samples (b can be temporarily imagined as θ_0 in the previous section).

$$L(\omega, b) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \sum_{i=1}^m (y_i - \omega^T x_i - b)^2$$

- To briefly sort it out, our goal in the training process (here we call the process of determining ω and b the training process) is to find the ω and b that cause $L(\omega, b) = \sum_{i=0}^m (y_i - \hat{y}_i)^2 = \sum_{i=0}^m (y_i - \omega^T x_i - b)^2$ to take the minimum value.
- And in the test process (where we call the process of computing $\hat{y}$ from the obtained ω and b), we input x and the output is $\hat{y} = \omega^T x + b$.

PS Linear regression

3. Determine the multiple linear regression model

Here, we try to solve for ω and b without using gradient descent, but using the **least squares method** abandoned in the previous section. for the sake of discussion, we set the **dimension of x to 1** in the following derivation, i.e., $d = 1$.

1) First, taking the **partial derivative** of the **loss function L** with ω and b respectively

Note: The **least squares** method, when used as a loss function, does **not** divide by the total number of samples m . The **mean squared error (MSE)** does divide by the total number of samples m . Please carefully distinguish between them in your future studies.

$$L(\omega, b) = \sum_{i=1}^m (y_i - \omega x_i - b)^2$$

$$\frac{\partial L(\omega, b)}{\partial b} = -2 \sum_{i=1}^m (y_i - \omega x_i - b)$$

$$\frac{\partial L(\omega, b)}{\partial \omega} = -2 \sum_{i=1}^m (x_i y_i - \omega x_i^2 - b x_i)$$

2) Let $\frac{\partial L(\omega, b)}{\partial b} = 0$ $b = \frac{1}{m} \sum_{i=1}^m (y_i - \omega x_i)$

3) $\because m\bar{y} = \sum_{i=1}^m y_i, m\bar{x} = \sum_{i=1}^m x_i$

$b = \bar{y} - \omega \bar{x} \rightarrow \frac{\partial L(\omega, b)}{\partial \omega} = ?$

4) $\frac{\partial L(\omega, b)}{\partial \omega} = -2 \left[\sum_{i=1}^m (x_i y_i - \omega x_i^2) - \bar{y} \sum_{i=1}^m x_i + \omega \bar{x} \sum_{i=1}^m x_i \right]$

PS Linear regression

3. Determine the multiple linear regression model

$$5) \text{Let } \frac{\partial L(\omega, b)}{\partial \omega} = 0 \quad -2 \left[\sum_{i=1}^m (x_i y_i - \omega x_i^2) - \bar{y} \sum_{i=1}^m x_i + \omega \bar{x} \sum_{i=1}^m x_i \right] = 0$$

$$\omega = \frac{\sum_{i=1}^m (x_i y_i) - \bar{y} \sum_{i=1}^m x_i}{\sum_{i=1}^m x_i^2 - \bar{x} \sum_{i=1}^m x_i} = \frac{\sum_{i=1}^m (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^m (x_i - \bar{x})^2}$$

4. If $d \neq 1$, Multiple linear regression

$$\hat{y} = \omega^T x + b$$

$$\hat{\omega} = (\omega; b), X = (x_1^T, 1; x_2^T, 1; \dots; x_m^T, 1) \rightarrow \hat{y} = X \hat{\omega}$$

$$L(\omega, b) = L(\hat{\omega}) = (y - X \hat{\omega})^T (y - X \hat{\omega})$$

Taking the **partial derivative** of the **loss function L** with $\hat{\omega}$

$$\frac{\partial L(\hat{\omega})}{\partial \hat{\omega}} = 2X^T(X\hat{\omega} - y)$$

* denotes a number other than 0

$$\text{Let } \frac{\partial L(\hat{\omega})}{\partial \hat{\omega}} = 0$$

$$\hat{\omega}^* = (X^T X)^{-1} X^T y$$

矩阵-求导公式

注: 对X求偏导

$$(X^T)' = I$$

$$(AX^T)' = A$$

$$(X^T A)' = A$$

$$(AX)' = A^T$$

$$(XA)' = A^T$$

$$(X^T A X)' = (A + A^T)X$$

$$(X^T A X)' = 2AX \quad (A \text{ 为对称矩阵})$$

矩阵-转置公式

$$(mA)^T = mA^T$$

$$(A + B)^T = A^T + B^T$$

$$(AB)^T = B^T A^T$$

$$(A^T)^T = A$$

If $X^T X$ is positive definite (i.e., full rank and all eigenvalues are greater than zero), then it is invertible, and we can solve for $\hat{\omega}$.

Linear regression is generally used in regression problems, in other words where the **output is continuous**.

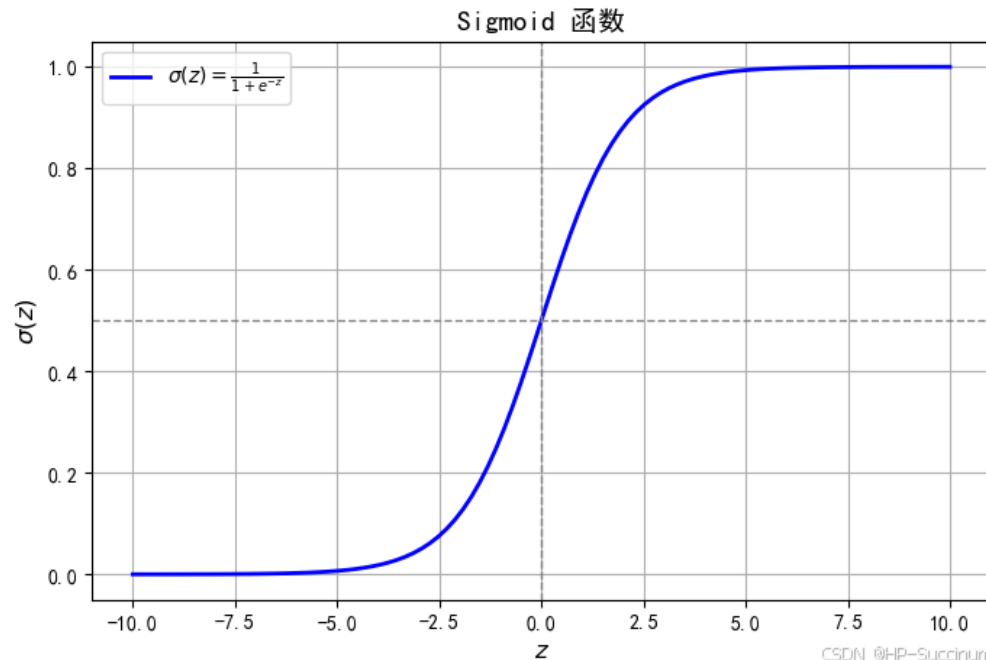
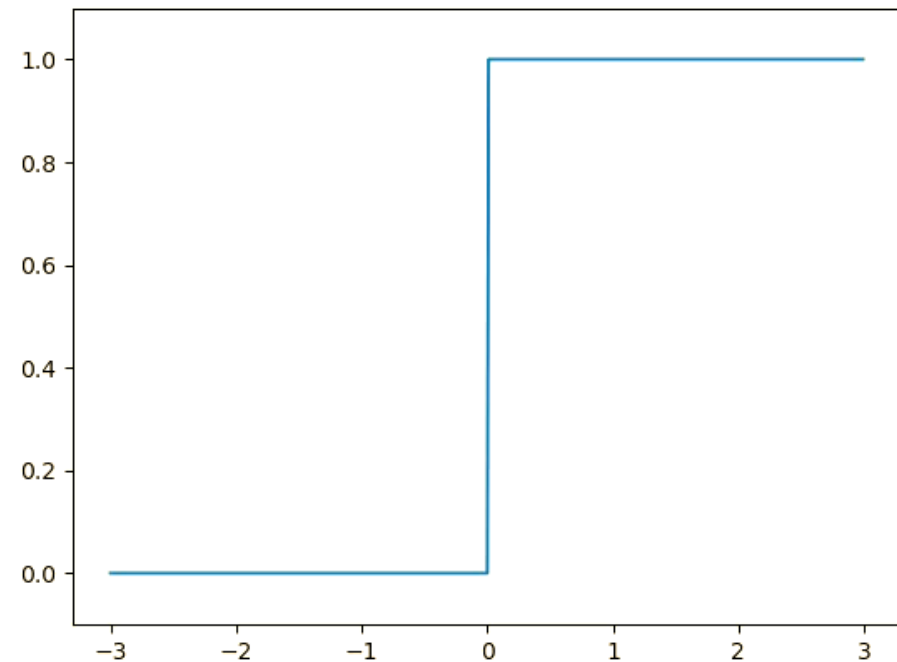
- The watermelon **book** begins with an analysis of linear regression, followed by an analysis of **how linear regression can be applied to classification problems**.
- Since the predictive structure of linear regression $\hat{y}$ is a real value, and if we are dealing with a **Binary Classification** problem, we **want** the **output to be 0 or 1**.
- Therefore it is necessary to find a function so that $f(\hat{y}) \in \{0,1\}$, and as a result the **step function** is chosen (output values are 0, 0.5, 1).
- However, the unit step function is not continuous, and it is **not easy to take partial derivatives** of the **parameters** when using the **gradient descent method**, so a series of operations such as **Logistic Regression** are chosen.

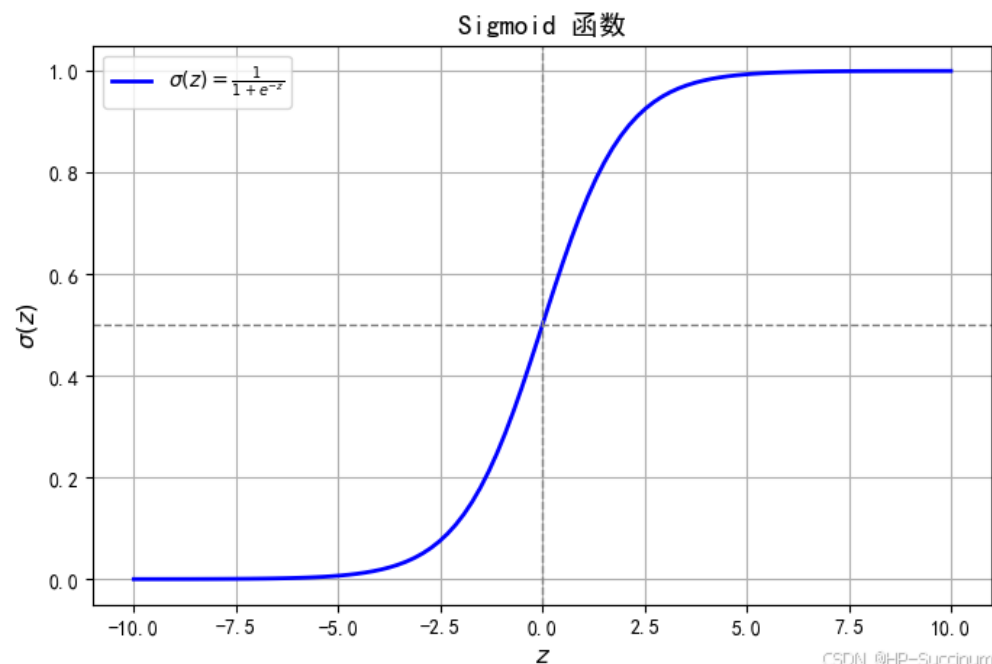
Note that in this **classification problem**, the goal is **not to minimize the error function, but to maximize the likelihood that each sample will be correctly classified**, for more details please refer to the watermelon book.

The **ERM** introduced in CS168 deal with the **Binary Classification** problem. The classification model gives a **discontinuous binary output with $h(x)$ taking the value 0 or 1**. The **training error is also based on the loss function 0-1 distribution**, which is different from the entry point of the watermelon book, so please have time to take a closer look at it in class.

PS Thinking: From Regression to Classification

In previous case studies, we explored **linear regression**, which **predicts continuous values by fitting a straight line**. However, in scenarios requiring **binary outcomes** (yes/no or 0/1)—a **binary classification problem**—directly applying **linear fitting with a threshold** often **fails** to model the data distribution effectively. While linear regression outputs values in $(-\infty, +\infty)$, binary classification operates within $[0, 1]$, and abrupt **step-like** transitions (resembling a staircase) may occur between classes.





We can address the 0-1 classification problem using **logistic regression**. Let's first examine the function curve shown in the figure below—the **Sigmoid function**, defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This curve exhibits a perfect **S-shape** (hence the name "**Sigmoid**"), with outputs ranging between **0** and **1**. It is centrally symmetric about the $z=0$ axis, making it inherently well-suited for representing probabilities.

PS Logistic regression

Logistic regression belongs to the **generalized linear model** family and is primarily used for **binary classification**. Unlike linear regression, logistic regression maps the results of linear regression to the (0,1) interval via the **Sigmoid function** (also called the logistic function), outputting a probability value that represents the likelihood of an event occurring. The formula is as follows:

$$P(Y = 1|X) = h_{\omega}(X) = \sigma(\omega^T X + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$P(Y = 1|X)$ represents the probability of a sample belonging to **class 1** given the input features X .

- X is the input feature vector: $X = (x_1, \dots, x_n)$
- ω is the weight vector of the features: $\omega = (\omega_1, \dots, \omega_n)$
- b is the bias term (intercept).

Decision Boundary – How Classification Works

The output of a logistic regression model is a **probability value**, indicating the likelihood of a sample belonging to a specific class. To make classification decisions, we typically set a **threshold** (e.g., 0.5):

- If the model's output probability is ≥ 0.5 , the sample is predicted as **class 1**.
- If the probability is < 0.5 , the sample is predicted as **class 0**.

The decision rule can be formalized as:

$$\hat{Y} = \begin{cases} 1, & \text{if } P(Y = 1|X) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$$

Loss Function of Logistic Regression

Derivation of the Loss Function:

The derivation leverages the **maximum likelihood estimation (MLE)** principle: given known X and Y (training set), we aim to find weights ω that maximize the probability of observing Y under the condition X .

- For binary classification ($Y = 1$ or $Y = 0$) the conditional probability is defined as:

$$P(Y|X; \omega) = \begin{cases} h_{\omega}(X), & Y = 1 \\ 1 - h_{\omega}(X), & Y = 0 \end{cases}$$

- This can be compactly expressed as:

$$P(Y|X; \omega) = (h_{\omega}(X))^Y (1 - h_{\omega}(X))^{1-Y}$$

- Assuming training samples are independent, the likelihood function becomes:

$$L(\omega) = \prod_{i=1}^n P(Y^{(i)}|X^{(i)}; \omega) = \prod_{i=1}^n (h_{\omega}(X^{(i)}))^{Y^{(i)}} (1 - h_{\omega}(X^{(i)}))^{1-Y^{(i)}}$$

Loss Function of Logistic Regression

- Taking the natural logarithm (base e):

$$\ell(\omega) = \ln L(\omega) = \ln\left(\prod_{i=1}^n (h_{\omega}(X^{(i)}))^{Y^{(i)}} (1 - h_{\omega}(X^{(i)}))^{1-Y^{(i)}}\right)$$

- Simplifying the product into a summation:

$$\ell(\omega) = \ln L(\omega) = \sum_{i=1}^n \left(Y^{(i)} \ln(h_{\omega}(X^{(i)})) + (1 - Y^{(i)}) \ln(1 - h_{\omega}(X^{(i)})) \right)$$

Summary:

We derive the log-likelihood expression for logistic regression. Next:

- ✓ construct the likelihood function.
- ✓ Apply **maximum likelihood estimation (MLE)**.
- ✓ Derive the iterative update rule for ω .

However, unlike linear regression (which minimizes a loss function via **gradient descent**), logistic regression **maximizes** the log-likelihood using **gradient ascent**.

Loss Function of Logistic Regression

Since loss functions are conventionally minimized, we negate the log-likelihood to convert the maximization problem into a minimization task. The final **loss function** becomes:

$$\begin{aligned} J(\omega) &= -\ell(\omega) = -\ln L(\omega) \\ &= -\sum_{i=1}^n \left(Y^{(i)} \ln(h_{\omega}(X^{(i)})) + (1 - Y^{(i)}) \ln(1 - h_{\omega}(X^{(i)})) \right) \end{aligned}$$

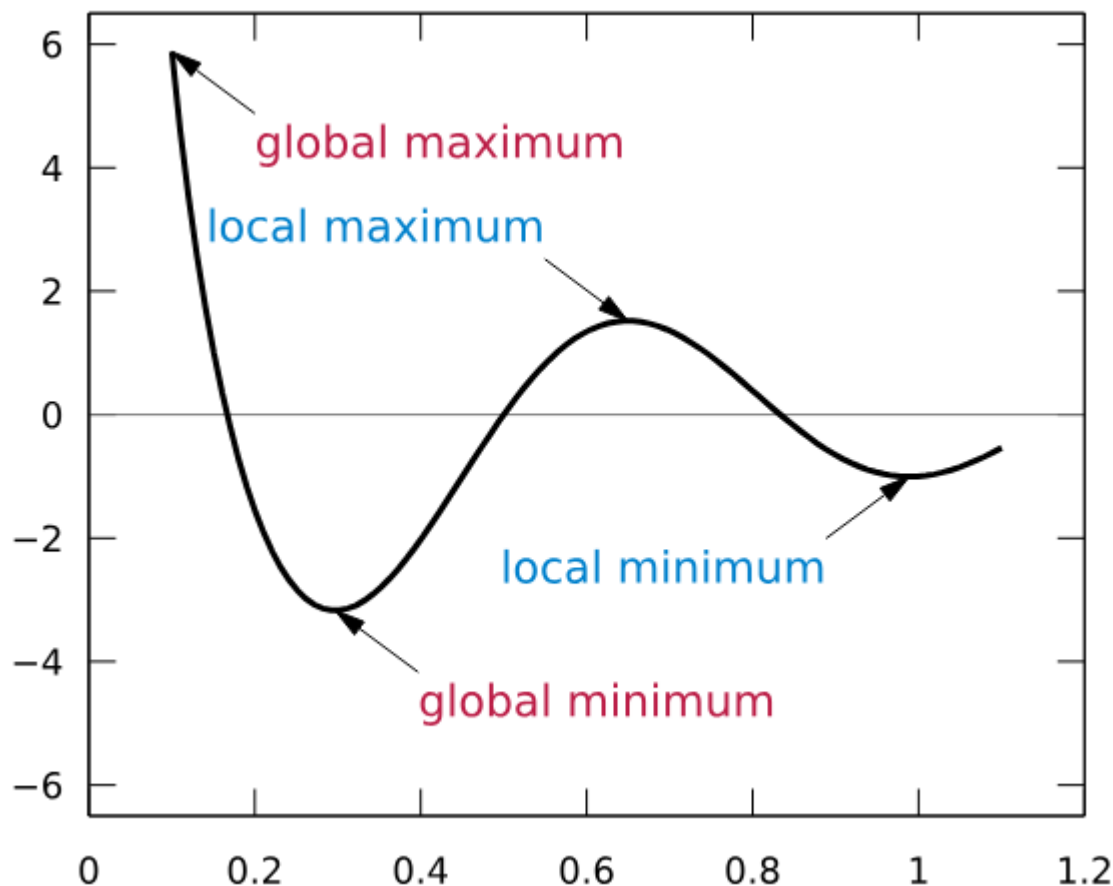
To measure the average loss across the entire dataset, we compute the mean over all samples, forming logistic regression's final **log-loss function**. It is expressed as:

$$J(\omega) = -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \ln(h_{\omega}(X^{(i)})) + (1 - Y^{(i)}) \ln(1 - h_{\omega}(X^{(i)})) \right)$$

Why Doesn't Logistic Regression Use the Squared Loss Function from Linear Regression?

The goal of defining a loss function is to find its **minimum** through optimization methods, where a smaller loss indicates a better model.

1. **Convex Functions** guarantee a **global minimum** (if it exists).
2. **Non-Convex Functions** often lead to **local minima**, making optimization unstable.



When applying the **squared loss function** (used in linear regression) to logistic regression, the resulting loss landscape becomes **non-convex** (as shown in the figure above).

Optimization: How to Train a Logistic Regression Model?

Logistic regression optimizes its parameters (weights ω and bias b) using **gradient descent**. This iterative algorithm updates the parameters to minimize the loss function. The update rules are:

$$\omega := \omega - \alpha \frac{\partial J}{\partial \omega}, \quad b := b - \alpha \frac{\partial J}{\partial b}$$

where:

- α is the **learning rate**, controlling the step size of parameter updates.

The gradients of the loss function are computed as follows:

$$h_{\omega}(X) = \sigma(\omega^T X + b) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

PS Optimization

- When taking the derivative of the logistic regression function, there is a property that will be used in the derivation below. This property is:

$$\begin{aligned}\sigma'(z) &= \frac{\partial}{\partial z} \frac{1}{1 + e^{-z}} \\ &= \frac{-e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{-e^{-z}}{(1 + e^{-z})^2} \cdot (1 + e^{-z}) \\ &= \frac{-e^{-z}}{1 + e^{-z}} \\ &= -\sigma(z) \cdot (1 - \sigma(z))\end{aligned}$$

PS Optimization

- Let's go back to the derivation of the logistic regression loss function:

$$J(\omega) = -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \ln(h_{\omega}(X^{(i)})) + (1 - Y^{(i)}) \ln(1 - h_{\omega}(X^{(i)})) \right)$$

$$\begin{aligned} \frac{\partial}{\partial \omega_j} J(\omega) &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \frac{1}{h_{\omega}(X^{(i)})} \frac{\partial}{\partial \omega_j} h_{\omega}(X^{(i)}) - (1 - Y^{(i)}) \frac{1}{1 - h_{\omega}(X^{(i)})} \frac{\partial}{\partial \omega_j} h_{\omega}(X^{(i)}) \right) \\ &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \frac{1}{h_{\omega}(X^{(i)})} - (1 - Y^{(i)}) \frac{1}{1 - h_{\omega}(X^{(i)})} \right) \frac{\partial}{\partial \omega_j} h_{\omega}(X^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \frac{1}{\sigma(\omega^T X + b)} - (1 - Y^{(i)}) \frac{1}{1 - \sigma(\omega^T X + b)} \right) \frac{\partial}{\partial \omega_j} \sigma(\omega^T X^{(i)} + b) \\ &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \frac{1}{\sigma(\omega^T X + b)} \right. \\ &\quad \left. - (1 - Y^{(i)}) \frac{1}{1 - \sigma(\omega^T X + b)} \right) \sigma(\omega^T X^{(i)} + b) \cdot (1 - \sigma(\omega^T X^{(i)} + b)) \frac{\partial}{\partial \omega_j} \omega^T X^{(i)} \\ &= -\frac{1}{m} \sum_{i=1}^n (Y^{(i)} (1 - \sigma(\omega^T X^{(i)} + b)) - (1 - Y^{(i)}) \sigma(\omega^T X^{(i)} + b)) X_j^{(i)} \end{aligned}$$

PS Optimization

$$\begin{aligned}\frac{\partial}{\partial \omega_j} J(\omega) &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} \left(1 - \sigma(\omega^T X^{(i)} + b) \right) - (1 - Y^{(i)}) \sigma(\omega^T X^{(i)} + b) \right) X_j^{(i)} \\ &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} - \sigma(\omega^T X^{(i)} + b) \right) X_j^{(i)} \\ &= -\frac{1}{m} \sum_{i=1}^n \left(Y^{(i)} - h_{\omega}(X^{(i)}) \right) X_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^n \left(h_{\omega}(X^{(i)}) - Y^{(i)} \right) X_j^{(i)}\end{aligned}$$

$$\omega_j := \omega_j - \alpha \frac{\partial J}{\partial \omega_j} = \omega_j - \alpha \frac{1}{m} \sum_{i=1}^n \left(h_{\omega}(X^{(i)}) - Y^{(i)} \right) X_j^{(i)}$$

Because α is originally a constant, $1/m$ is generally omitted. Therefore, the final θ update process is:

$$\omega_j := \omega_j - \alpha \frac{\partial J}{\partial \omega_j} = \omega_j - \alpha \sum_{i=1}^n \left(h_{\omega}(X^{(i)}) - Y^{(i)} \right) X_j^{(i)}$$

4. 偏置项 b 的梯度

对于偏置 b , 损失函数为:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_w(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)})) \right].$$

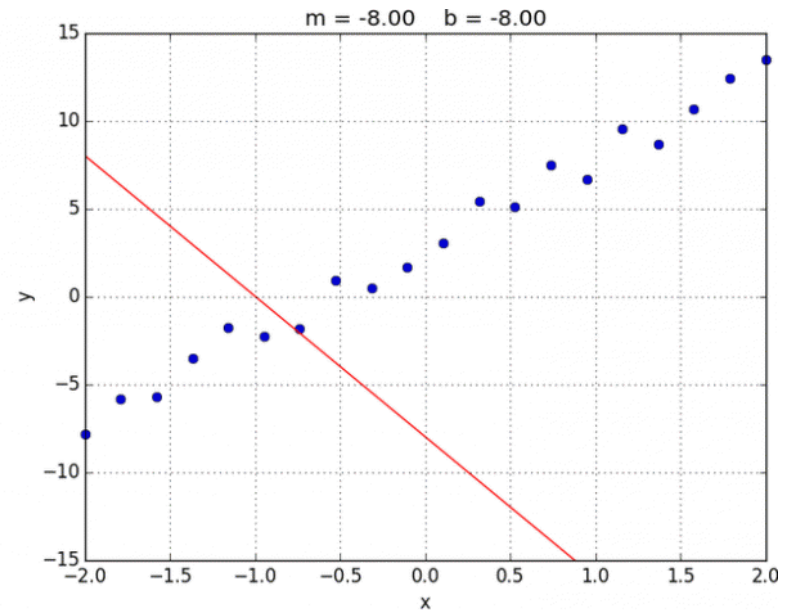
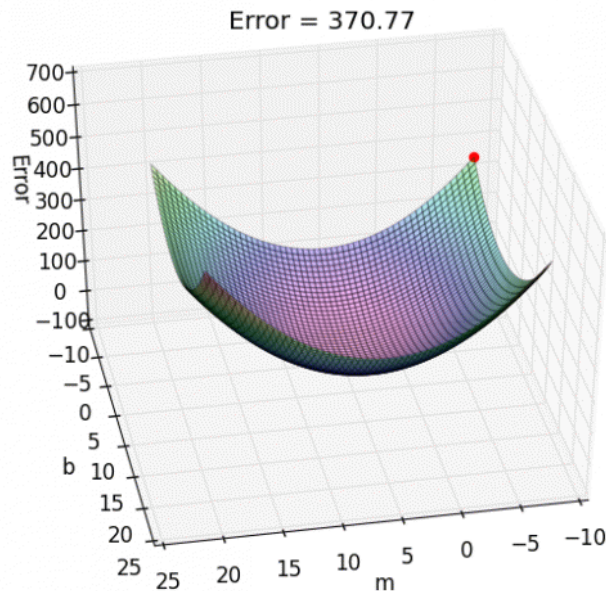
对 b 求导:

$$\frac{\partial J(w, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}).$$

$$b := b - \alpha \frac{\partial J}{\partial b} = b - \alpha \sum_{i=1}^n (h_{\omega}(X^{(i)}) - Y^{(i)})$$

[Keywords] Logistic function, Maximum Likelihood Estimation, Gradient Descent Method

- (1) Find an appropriate prediction function h . This is the classification function we need, which is used to predict the outcome of the input data.
- (2) Construct a cost function (loss function). This function represents the deviation between the predicted output (h) and the actual class label (y) of the training data. Considering the "loss" of all training data, sum or average the costs to obtain the $J(\theta)$ function, which represents the deviation between the predicted values and the actual classes for all training data.
- (3) Obviously, the smaller the value of the $J(\theta)$ function, the more accurate the prediction function h . Therefore, the task in this step is to find the minimum value of the $J(\theta)$ function. In Logistic Regression, the Gradient Descent method is used.



PS Follow the previous and start the next

It seems that in the **empirical risk minimization** process, where we try to **reduce the difference between the predicted and true values of the model**, we are solving the problem of **excessive bias**.



Then next, in the **regularization (structural risk minimization)**, we have to start thinking about solving the **excessive variance** (**overfitting**, overly complex prediction models for **test set** output oscillations).

◆ **Reducing Model Complexity**

◆ **Regularization**

Always keep in mind that deep learning models are usually very good at fitting training data, but the **real challenge is generalization**, not fitting.

3.6 Structural Risk Minimization

However, in reality, **the number of training samples is limited**, and using **empirical risk** to estimate the expected risk is often **not ideal**, and we have to provide some correction for empirical risk (**adding a penalty term**, which serves to **simplify the model**) is required to form the **structural risk**.

Structural risk minimization (SRM)

A strategy proposed to **prevent overfitting**. Structural risk minimization is equivalent to **regularization**.

$$R_{srm}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

$\lambda \geq 0$, used to weigh empirical risk and model complexity

Structural risk **adds** to the empirical risk a **regularizer or penalty term** that represents the **complexity** of the model.

A function representing the complexity of the model

The **structural risk minimization** strategy considers the model with the **least structural risk** as the optimal model

$$\min_{f \in F} \left[\frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f) \right]$$

Low structural risk requires **both empirical risk and model complexity to be small**, and models with **low structural risk** tend to predict **training data** as well as unknown **test data** better.

3.6 Regularization

?

As mentioned above:

- The **goal** of regularization is to **prevent overfitting**
 - By means of **reducing the complexity of the model**
 - The leverage is λ , by which we **adjust the weights** of the **penalty terms**, thus "polishing" our prediction model to **make it smooth, but not too smooth**.
-
- ✓ Recalling the composition of the penalty term, we find that:
 - ✓ The **penalty term** is a **function of the parameter w** (λ is a constant),
 - ✓ and the process of minimizing the structural risk must try to touch the **minimum value of this function**
 - ✓ so the core is to **reduce the value of each parameter** (w in the video) while **keep all parameters present**
 - ✓ so that the predicted output is not overfitted because the parameter value w is too large, with the independent variable x "Overreacting".

3.6 Regularization & norm

?

Recall the various forms of Norm

2.3 Norm

Norm of vectors

indicates the size of the original set

l_0 -norm: If the vector $\vec{x}$ is $(x_1, x_2, \dots, x_n)$, **l_0 norm** of $\vec{x}$ is

$$\|\vec{x}\|_0$$

= *Number of all non-zero elements in $\vec{x}$*

If vector $\vec{a} = (0, 3, 6)$, $\vec{a}$ has 1 element 0 and 2 non-zero elements (3 and 6), then corresponds to **l_0 -norm** $\|\vec{a}\|_0 = 2$

l_1 -norm: If the vector $\vec{x}$ $(x_1, x_2, \dots, x_n)$, **l_1 norm** of $\vec{x}$ is

$$\|\vec{x}\|_1$$

= *sum of the absolute values of all elements in $\vec{x}$*

$$= |x_1| + |x_2| + \dots + |x_n| = \sum_{i=1}^n |x_i|$$

If vector $\vec{a} = (0, 3, 6)$, corresponds to **l_1 -norm** $\|\vec{a}\|_1 = |0| + |3| + |6| = 9$

L1 and L2 norm are mainly used as loss functions and regular terms in machine learning

l_2 -norm: for vector $\vec{x}$ $(x_1, x_2, \dots, x_n)$

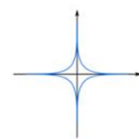
$$\|\vec{x}\|_2 = \|\vec{x}\|$$

= *sum of the squares of the absolute values of all elements in $\vec{x}$, and then find square root*

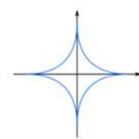
$$= \sqrt{\sum_{i=1}^n |x_i|^2}$$



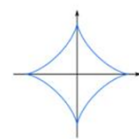
$$p = 2^{-2} \\ = 0.25$$



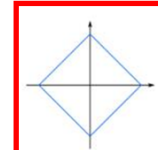
$$p = 2^{-1.5} \\ = 0.354$$



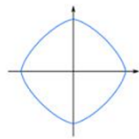
$$p = 2^{-1} \\ = 0.5$$



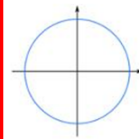
$$p = 2^{-0.5} \\ = 0.707$$



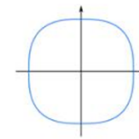
$$p = 2^0 \\ = 1$$



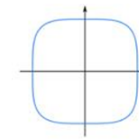
$$p = 2^{0.5} \\ = 1.414$$



$$p = 2^1 \\ = 2$$

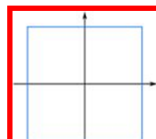


$$p = 2^{1.5} \\ = 2.828$$



$$p = 2^2 \\ = 4$$

...



$$p = 2^\infty \\ = \infty$$

3.6 Regularization & norm

L1 regularity and **L2 regularity** are the **commonly** used regularization methods

L1 can generate a sparse weight matrix, i.e., a **sparse model** that can be used for **feature selection**, and can **prevent overfitting**

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (y^i - h_{\theta}(x^i))^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

The L1-norm is the sum of the absolute values of the elements of the pointing quantity, and the L1-norm is the **optimal convex approximation** of the L0-norm.

$$\theta_j := \theta_j - \alpha \frac{\lambda}{m} \text{sgn}(\theta_j) - \alpha \frac{1}{m} \sum_{i=1}^m (y^i - h_{\theta}(x^i)) x_j^i$$

$$\text{sgn}(\theta_j) = \begin{cases} 1, & \theta_j > 0 \\ 0, & \theta_j = 0 \\ -1, & \theta_j < 0 \end{cases}$$



Each round of training θ_j moves closer to 0, and eventually an approximate sparse solution can be obtained

L2 prevent model **overfitting**

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (y^i - h_{\theta}(x^i))^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

where m is the number of samples and n is the number of features. In order to **minimize the loss function**, the estimates are obtained by taking **partial derivatives** of each model parameter and repeat

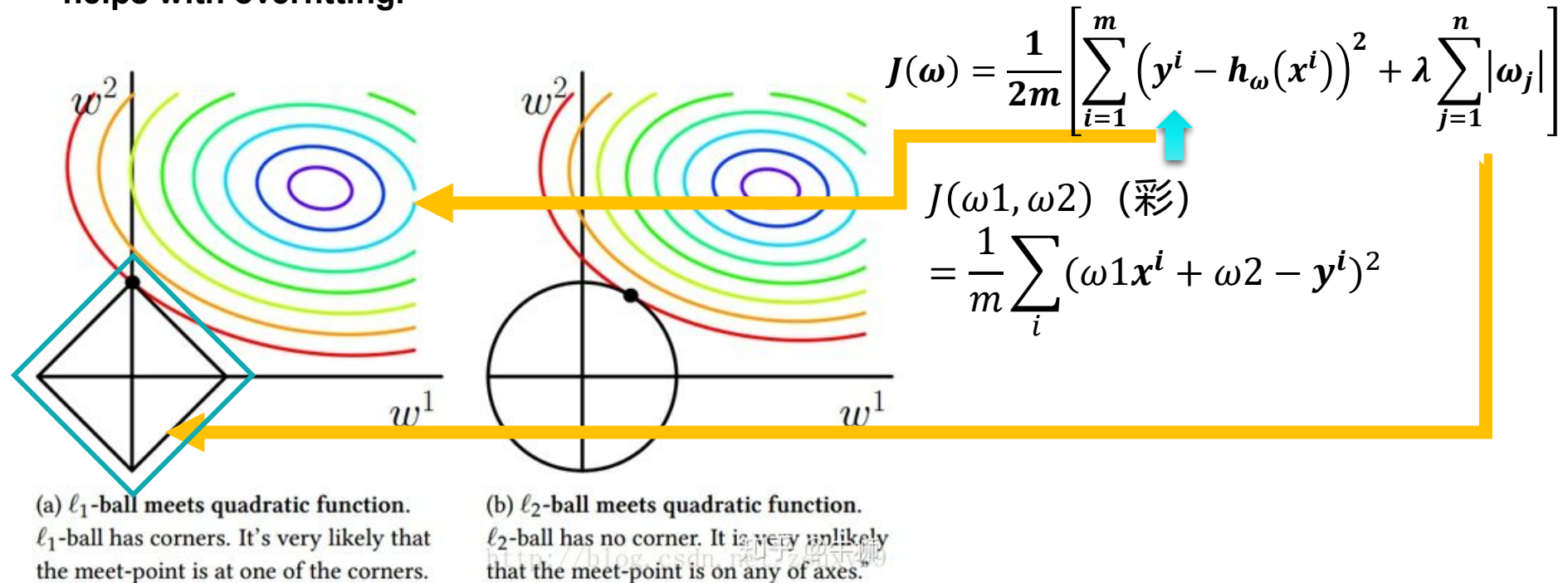
$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

$$\theta_j := \theta_j (1 - \alpha \frac{\lambda}{m}) - \alpha \frac{1}{m} \sum_{i=1}^m (y^i - h_{\theta}(x^i)) x_j^i$$

α is the step size, $0 < 1 - \alpha \frac{\lambda}{m} < 1$, so the L2 regularity scales down each model parameter θ_j to some degree, **but not to 0**

PS optimal convex approximation

- Using this figure, let's examine how the **regularization** term **can take a smaller value** when **the empirical risk is fixed**. Suppose we are aiming at the red circle — where the empirical risk is constant — how can the penalty term be made relatively small?
- That is, find the smallest diamond that still intersects the red circle. The diamond first touches the red circle with its sharp corner. Although the blue diamond also intersects the red circle, it is larger (resulting in a larger penalty term).
- So, what we want to find is the **black diamond**.
- On the black diamond, $\omega_1=0$. This is why we say that **L1 regularization yields sparse solutions**.
- In contrast, **L2 regularization** corresponds to the intersection of circles (or a circle with a circle). The intersection point does not lie on the coordinate axes, so it cannot produce sparse solutions — **it only helps with overfitting**.



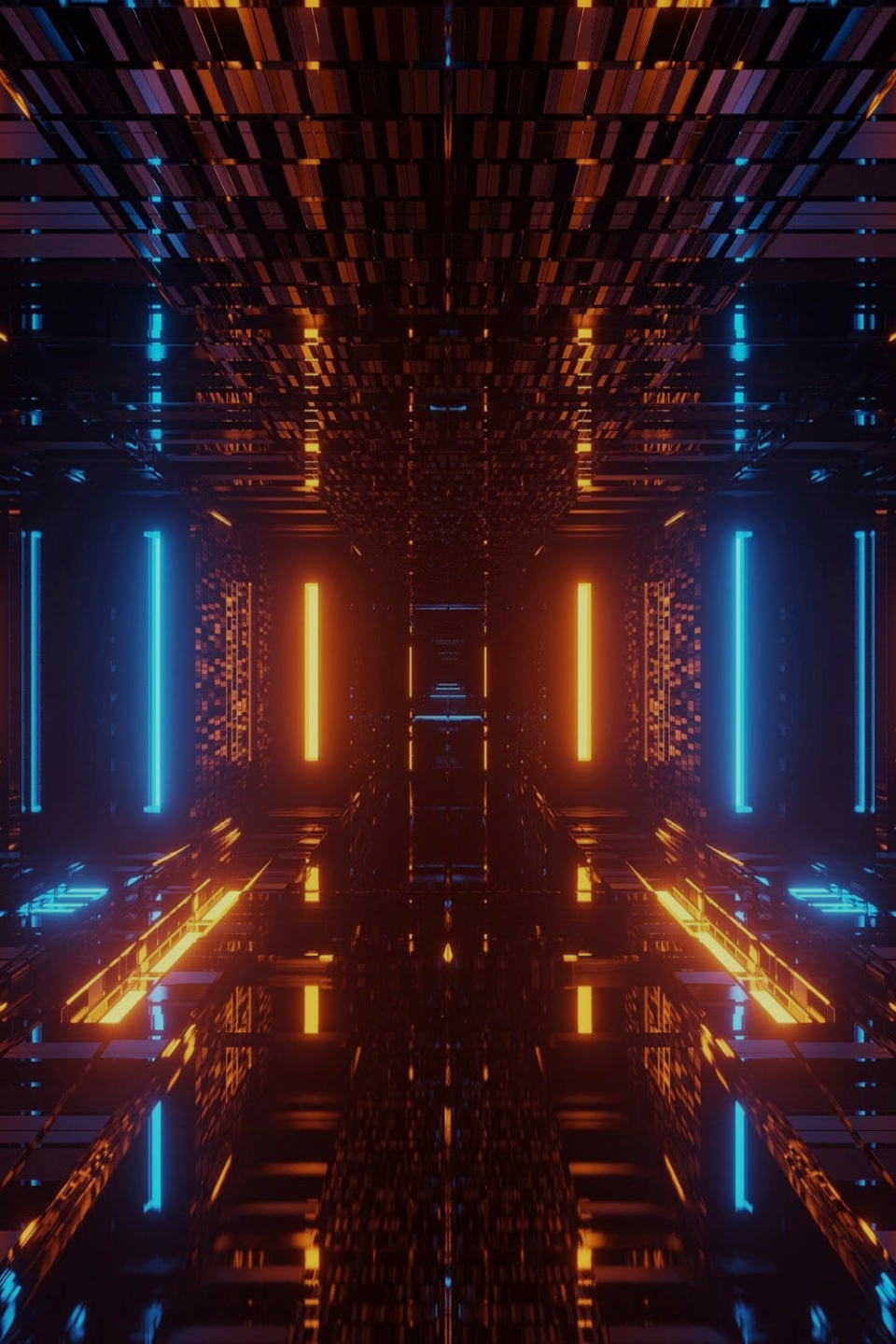
Research Assignment:

Submit the report to luoxsince1989@qq.com by **June 16**.

Topic: Spectral Clustering

Requirements:

1. Explain what clustering is, and list **at least 6 clustering algorithms** (include **scatter plots**).
2. Provide the **definition** of spectral clustering and analyze its core idea. Attempt to **compare spectral clustering** with **K-means**, and illustrate the advantages of spectral clustering with examples (scatter plots may be used).
3. A spectral clustering algorithm typically consists of five major steps: graph construction, building the graph, spectral decomposition, clustering in the new space, and result mapping. With the help of diagrams, analyze in detail **how the relationship network is built in spectral clustering, the form of the Laplacian matrix** and the use of its **eigenvectors**, as well as the process of **dimensionality reduction and clustering**.
4. Explain how spectral clustering can be **used in social network analysis**, for example, to automatically identify friend circles, etc.
5. **The entire research report shall be no less than 2500 words.**



Thanks